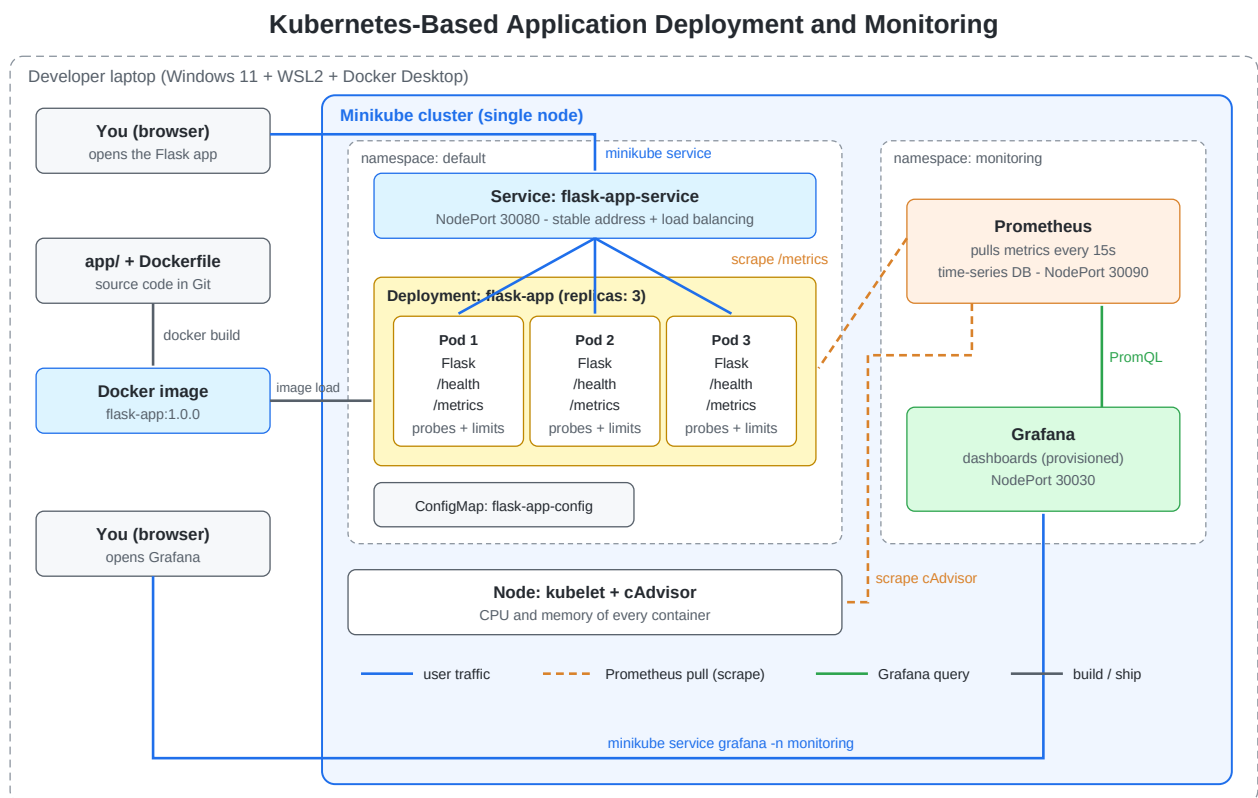


Kubernetes-Based Application Deployment and Monitoring

Docker · Minikube · Prometheus · Grafana · Python Flask

Study guide: build, operate, monitor, troubleshoot and explain the project



Radhika Agarwal

October 2026

Contents

Phase-by-Phase Build Guide	3
Phase 1 - Architecture and Prerequisites	3
Phase 2 - The Flask Application	6
Phase 3 - Dockerize the Application	7
Phase 4 - Minikube and the Kubernetes Environment	8
Phase 5 - Deployment and Service YAML	9
Phase 6 - Replicas, Probes and Resource Limits	11
Phase 7 - Prometheus and Grafana	12
Phase 8 - The Grafana Dashboard	14
Phase 9 - Testing and Troubleshooting	15
Phase 10 - GitHub Repository and README	17
Troubleshooting	19
The method: get, describe, logs, events, exec	19
Error catalogue	20
Useful one-liners	21
Interview Preparation	22
The 60-second project pitch	22
Core questions	22
Follow-up questions to be ready for	24
Resume-ready description	24
Appendix - Source Files	25

Phase-by-Phase Build Guide

This guide rebuilds the project from scratch, one phase at a time. Every command is explained in the same format:

- **What** - what the command does
- **Why** - why the project needs it
- **Where** - which terminal and folder to run it in
- **Expected** - what a successful result looks like
- **If it fails** - the common errors and how to fix them

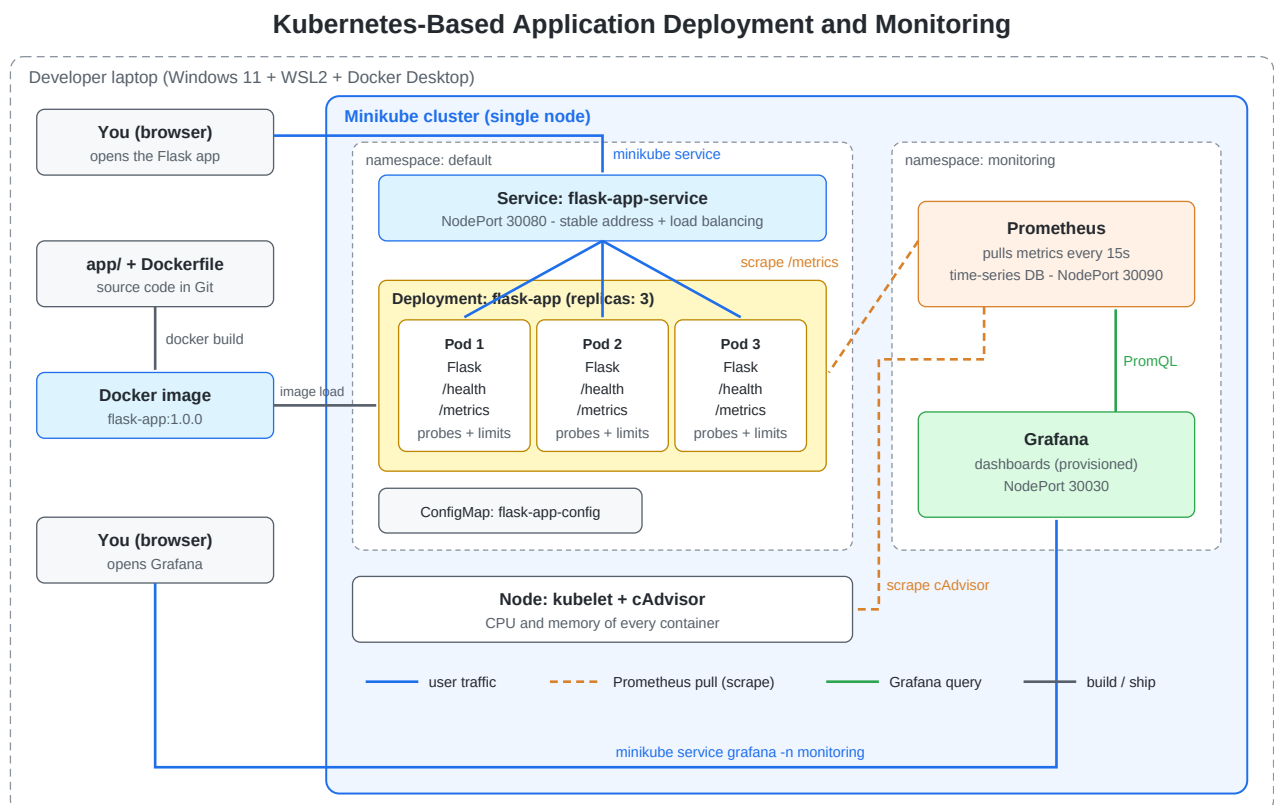
Terminals used in this guide:

Name	How to open it	Used for
Power Shell	Start menu, type "PowerShell"	Installing tools on Windows
Git Bash	Right-click in the project folder, "Open Git Bash here"	Every <code>docker</code> , <code>minikube</code> , <code>kubectl</code> and script command

All commands except the Phase 1 installs are run from the project root folder (`k8s-flask-monitoring/`). The same commands work on Linux and macOS.

Phase 1 - Architecture and Prerequisites

The big picture



1. You write a small **Flask** app and package it into a **Docker image**.

2. The image is loaded into **Minikube**, a one-node Kubernetes cluster that runs inside Docker on your laptop.
3. A **Deployment** keeps 3 copies (pods) of the app running. A **Service** gives them one stable address and spreads traffic across them.
4. **Prometheus** pulls (scrapes) metrics every 15 seconds: CPU and memory from **cAdvisor** (part of the kubelet), and request counts from the app's `/metrics` endpoint.
5. **Grafana** queries Prometheus and draws the dashboard.

Why each tool is here

Tool	Problem it solves
Docker	"It works on my machine": the image carries Python, the libraries and the code together
Kubernetes	Running many containers reliably: restarts, scaling, load balancing, rolling updates
Minikube	Real Kubernetes on a laptop without paying for a cloud cluster
Prometheus	Collects and stores numbers over time (time-series database)
Grafana	Turns those numbers into graphs humans can read

How DRY (Don't Repeat Yourself) is applied

Fact	Single source of truth	Who reuses it
App port, name, version	<code>app/config.py</code> (env vars with defaults)	Flask, gunicorn
Image tag / version	<code>image:</code> line in <code>k8s/deployment.yaml</code>	<code>scripts/deploy.sh</code> reads it and passes it to <code>docker build</code>
Port number 5000	<code>containerPort</code> named <code>http</code>	Probes, Service <code>targetPort</code> , Prometheus scrape config all say <code>http</code>
Health check	YAML anchor <code>&health-check</code>	Readiness and liveness probes
Pod label <code>app: flask-app</code>	Deployment template	Service selector, tests
Dashboard	<code>monitoring/dashboards/flask-app.json</code>	Turned into a ConfigMap by the deploy script, never copied
Explanations	<code>docs/*.md</code>	README links to them, the PDF is generated from them

Prerequisites

Tool	Check command	Install (Windows)
WSL2	<code>wsl --status</code>	<code>wsl --install</code> (admin PowerShell, then reboot)
Docker Desktop	<code>docker version</code>	<code>winget install -e --id Docker.DockerDesktop</code>
Minikube	<code>minikube version</code>	<code>winget install -e --id Kubernetes.minikube</code>
kubectl	<code>kubectl version --client</code>	<code>winget install -e --id Kubernetes.kubectl</code>
Git + Git Bash	<code>git --version</code>	<code>winget install -e --id Git.Git</code>
Python 3.10+	<code>python --version</code>	<code>winget install -e --id Python.Python.3.12</code>

Hardware: at least 8 GB RAM (16 GB is comfortable) and virtualization enabled in the BIOS.

Install WSL2

```
wsl --install
```

- **What:** Installs the Windows Subsystem for Linux (version 2) and Ubuntu.
- **Why:** Containers are a Linux kernel feature (namespaces and cgroups). Docker Desktop runs its engine inside WSL2.
- **Where:** PowerShell opened with "Run as administrator".
- **Expected:** The requested operation is successful. Changes will not be effective until the system is rebooted. Reboot, then create a Linux username and password when Ubuntu opens.
- **If it fails:** 0x80370102 means virtualization is disabled. Enable Intel VT-x or AMD-V/SVM in the BIOS (Task Manager > Performance > CPU shows "Virtualization: Enabled" when it is on). "Requires elevation" means PowerShell was not opened as administrator. If the install hangs at 0%, use `wsl --install --web-download`.

Install and verify Docker Desktop

```
winget install -e --id Docker.DockerDesktop
```

- **What:** Installs Docker Desktop using the Windows package manager (`-e` = exact id match).
- **Why:** Docker builds the image, and Minikube uses Docker as its driver (the whole Kubernetes node is one Docker container).
- **Where:** PowerShell. Afterwards start Docker Desktop, accept the terms, and wait for "Engine running".
- **Expected:** Successfully installed.
- **If it fails:** If `winget` is not recognised, download the installer from docker.com.

```
docker run hello-world
```

- **What:** Downloads a tiny test image and runs it.
- **Why:** Proves the Docker client can talk to the Docker engine and pull images.
- **Where:** A new Git Bash window (new windows pick up the updated PATH).
- **Expected:** Hello from Docker! This message shows that your installation appears to be working correctly.
- **If it fails:** error during connect ... dockerDesktopLinuxEngine means Docker Desktop is not running. Start it and wait for the green "Engine running" status.

Create the repository with YOUR identity

```
mkdir -p ~/projects/k8s-flask-monitoring && cd ~/projects/k8s-flask-monitoring
git init -b main
git config user.name "Your Name"
git config user.email "you@example.com"
```

- **What:** Creates the folder, turns it into a Git repository, and sets the author for commits in this repository only.
- **Why:** On a shared laptop the global identity may belong to someone else. GitHub credits commits by email, so your email must be the one on your GitHub account.
- **Where:** Git Bash.
- **Expected:** Initialized empty Git repository in .../k8s-flask-monitoring/.git/
- **If it fails:** If commits later show the wrong person on GitHub, check `git log --format='%an <%ae>'` and make sure that email is verified on your GitHub account.

Phase 2 - The Flask Application

Files

File	Purpose
<code>app/config.py</code>	All settings in one place, read from environment variables with defaults
<code>app/app.py</code>	The routes and the Prometheus metrics
<code>app/gunicorn.conf.py</code>	Production web server settings (port comes from <code>config.py</code>)
<code>app/requirements.txt</code>	Pinned runtime dependencies
<code>tests/test_app.py</code>	Unit tests for the endpoints

Endpoints

Endpoint	Used by	Why it exists
<code>GET /</code>	People	Homepage. Shows which pod answered, so you can see load balancing
<code>GET /health</code>	Kubernetes probes	Returns 200 when healthy, 503 after <code>/fail</code>
<code>GET /metrics</code>	Prometheus	Request counters and latency histograms in Prometheus text format
<code>GET /work?ms=100</code>	Load tests	Burns CPU for a few milliseconds so the graphs move
<code>POST /fail</code>	Phase 9 demos	Makes <code>/health</code> fail, to watch probes in action

Key ideas in the code

- **Configuration from the environment (12-factor app):** The same image runs everywhere. Kubernetes changes behaviour by setting env vars (from the ConfigMap), not by rebuilding.
- **Metrics with labels:** `flask_http_requests_total{method, endpoint, status}` is a counter. The `endpoint` label uses the route pattern (`/work`), never the raw URL. Otherwise every different URL would create a new time series and could overload Prometheus ("high cardinality").
- **Counter vs Histogram:** A counter only goes up (total requests). A histogram puts durations into buckets so Prometheus can calculate percentiles such as p95 latency.
- **One gunicorn worker per container:** Kubernetes scales by adding pods, not processes. It also keeps the metrics in one process.

Run the tests

```
python -m venv .venv
source .venv/Scripts/activate      # Linux/macOS: source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest
```

- **What:** Creates an isolated Python environment, installs Flask and the test tools into it, then runs every test in `tests/`.
- **Why:** A virtual environment keeps project libraries separate from system Python. Tests prove the app works before it goes anywhere near Docker.
- **Where:** Git Bash, project root.
- **Expected:** 10 passed.
- **If it fails:** `ModuleNotFoundError: flask` means the venv is not active (the prompt should start with `(.venv)`). If `python` is not recognised, reinstall Python with "Add to PATH" ticked.

Run the app locally

```
python app/app.py
```

- **What:** Starts Flask's development server on port 5000.
- **Why:** The quickest feedback loop while writing code. (Gunicorn is used in the container. It does not run on Windows, and it does not need to here.)
- **Where:** Git Bash, project root, venv active. Leave it running and use a second terminal or a browser.
- **Expected:** Running on `http://127.0.0.1:5000`. Open `http://localhost:5000` and `http://localhost:5000/health`.
- **If it fails:** Address already in use means another program uses port 5000. Run `PORT=5050 python app/app.py` (the port comes from `config.py`).

```
curl localhost:5000/health
curl localhost:5000/metrics | grep flask_http
```

- **Expected:** `{"pod": "localhost", "status": "ok"}`, then lines such as `flask_http_requests_total{endpoint="/health",method="GET",status="200"} 1.0`.

Stop the server with `Ctrl+C`.

Phase 3 - Dockerize the Application

The Dockerfile, line by line

Instruction	Meaning
<code>FROM python:3.12-slim</code>	Start from the official Python image, Debian "slim" (small) variant
<code>ARG APP_VERSION=dev</code>	A build-time variable; the deploy script passes the real version
<code>ENV PYTHONUNBUFFERED=1 ...</code>	Print logs immediately so <code>kubectl logs</code> shows them in real time
<code>WORKDIR /app</code>	All following commands run in <code>/app</code> inside the image
<code>COPY app/requirements.txt .</code> then <code>RUN pip install</code>	Copy only the dependency list first so this slow layer is cached until requirements change
<code>COPY app/ .</code>	Copy the source code (changes often, so it comes last)
<code>RUN useradd ... /USER 10001</code>	Run as a normal user, not root, to limit damage if the app is compromised
<code>EXPOSE 5000</code>	Documentation of the port; it does not publish anything
<code>CMD ["gunicorn", ...]</code>	The process the container runs

Layer caching: each instruction creates a layer. Docker reuses a layer if nothing it depends on has changed. Putting rarely-changing steps first makes rebuilds take seconds.

`.dockerignore` keeps `.git`, `.venv`, tests and docs out of the build context, so the build is faster and the image smaller.

Build the image

```
docker build -t flask-app:1.0.0 .
```

- **What:** Reads the `Dockerfile` in the current folder (`.` is the build context) and creates an image tagged `flask-app:1.0.0`.
- **Why:** The image is the unit Kubernetes runs.

- **Where:** Git Bash, project root, Docker Desktop running.
- **Expected:** Several [n/7] steps ending with `naming to docker.io/library/flask-app:1.0.0`.
- **If it fails:** `failed to read dockerfile` means you are not in the project root. `Cannot connect to the Docker daemon` means Docker Desktop is not running. Network errors during `pip install` are usually a proxy or VPN.

```
docker images flask-app
```

- **Expected:** One row `flask-app 1.0.0 <id> ... ~150MB`.

Run the container

```
docker run -d --name flask-test -p 5000:5000 flask-app:1.0.0
curl localhost:5000/health
docker logs flask-test
docker rm -f flask-test
```

- **What:** `-d` runs in the background, `--name` gives it a name, `-p 5000:5000` maps laptop port 5000 to container port 5000. Then we check health, read the logs and remove the container.
- **Why:** Prove the image works on its own before adding Kubernetes. If it fails here, it will fail in Kubernetes too.
- **Where:** Git Bash.
- **Expected:** `curl` returns `"status": "ok"` with a container id as `pod`. `docker logs` shows `Listening at: http://0.0.0.0:5000` and the request line.
- **If it fails:** `port is already allocated` means something else uses 5000; use `-p 5050:5000`. If the container exits at once, `docker logs flask-test` shows the Python error.

Phase 4 - Minikube and the Kubernetes Environment

Start the cluster

```
minikube start --driver=docker --cpus=2 --memory=4096
```

- **What:** Creates a single-node Kubernetes cluster inside a Docker container with 2 CPUs and 4 GB RAM, and points `kubectl` at it.
- **Why:** Gives us a real Kubernetes API server, scheduler and kubelet to deploy to.
- **Where:** Git Bash, Docker Desktop running. The first run downloads about 1 GB.
- **Expected:** Ends with `Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default`.
- **If it fails:** `PROVIDER_DOCKER_NOT_RUNNING` means you should start Docker Desktop. If the error says Docker has less memory than requested, use `--memory=3072` or give Docker more memory in Settings > Resources. If an old broken cluster exists, run `minikube delete` and start again.

```
minikube status
kubectl get nodes
```

- **What:** Shows the state of the cluster components, then lists the cluster's nodes.
- **Expected:** `host: Running, kubelet: Running, apiserver: Running`, then one node `minikube Ready control-plane`.

- **If it fails:** The connection to the server ... was refused means the cluster is stopped (`minikube start`) or `kubectl` points elsewhere. Check with `kubectl config current-context`; it should print `minikube`.

Enable metrics-server

```
minikube addons enable metrics-server
```

- **What:** Installs the component that powers `kubectl top`.
- **Why:** A quick CPU and memory check from the terminal while troubleshooting. Prometheus is for history; `kubectl top` is for "right now".
- **Expected:** The 'metrics-server' addon is enabled. After about a minute, `kubectl top nodes` shows numbers.

Make the image available inside Minikube

```
minikube image load flask-app:1.0.0
minikube image ls | grep flask-app
```

- **What:** Copies the image from your Docker into Minikube's own container runtime.
- **Why:** Minikube has a separate image store. Without this, Kubernetes would try to download `flask-app:1.0.0` from Docker Hub and fail.
- **Expected:** `docker.io/library/flask-app:1.0.0`.
- **If it fails:** If it is not listed, rebuild the image (Phase 3) and load it again. Every time you rebuild, load again or change the tag.

Phase 5 - Deployment and Service YAML

Anatomy of every Kubernetes YAML file

Field	Meaning
<code>apiVersion</code>	Which API group and version defines this kind (<code>v1</code> , <code>apps/v1</code>)
<code>kind</code>	The type of object: Deployment, Service, ConfigMap...
<code>metadata</code>	Name, namespace and labels
<code>spec</code>	The desired state: what you want Kubernetes to make true

Kubernetes is **declarative**: you describe the desired state, and controllers keep working until the actual state matches it.

Pod vs Deployment vs Service

Object	What it is	Analogy
Pod	One or more containers sharing a network IP; temporary	One worker
Replica Set	Keeps exactly N pods matching a label	A supervisor who counts workers
Deployment	Manages ReplicaSets: rolling updates, rollback	The manager who hires and swaps supervisors
Service	Stable virtual IP and DNS name that load-balances across ready pods with a label	The reception desk phone number

The three files

- `k8s/configmap.yaml` - environment variables (`APP_ENV`) kept outside the image.
- `k8s/deployment.yaml` - 3 replicas of `flask-app:1.0.0`. `selector.matchLabels` must match `template.metadata.labels`; this is how the Deployment knows which pods it owns. `imagePullPolicy: IfNotPresent` uses the image we loaded. The port is named `http`.
- `k8s/service.yaml` - a `NodePort` Service. `selector: app: flask-app` picks the pods, `port: 80` is the in-cluster port, `targetPort: http` is the container port, and `nodePort: 30080` is opened on the node.

Service types: `ClusterIP` (inside the cluster only, the default), `NodePort` (also on every node's IP at a port from 30000-32767), and `LoadBalancer` (cloud load balancer). `NodePort` is the simplest choice that is reachable from outside on Minikube.

Deploy

```
kubectl apply -f k8s/
```

- **What:** Sends every YAML file in `k8s/` to the API server. `apply` creates objects or updates them to match the file.
- **Why:** This is the declarative workflow: the files in Git are the source of truth.
- **Where:** Git Bash, project root.
- **Expected:** `configmap/flask-app-config` created, `deployment.apps/flask-app` created, `service/flask-app-service` created. Running it again prints `unchanged`.
- **If it fails:** `error validating data` means a YAML indentation or field-name typo; the message names the line.

```
kubectl get pods -l app=flask-app -o wide
```

- **What:** Lists pods with the label `app=flask-app`, with their IP and node (`-o wide`).
- **Expected:** After 10-20 seconds, three pods `flask-app-<hash>-<id>` with `READY 1/1` and `STATUS Running`.
- **If it fails:** See the status table below, and the Troubleshooting chapter.

Status	Meaning	Fix
<code>ErrImagePull / ImagePullBackOff</code>	Image not found in Minikube	<code>minikube image load flask-app:1.0.0</code> ; check the tag matches the YAML
<code>CreateContainerConfigError</code>	Referenced ConfigMap or Secret missing	<code>kubectl apply -f k8s/configmap.yaml</code>
<code>CrashLoopBackOff</code>	Container starts and exits repeatedly	<code>kubectl logs <pod> --previous</code>
<code>Pending</code>	No node has enough free CPU or memory for the requests	<code>kubectl describe pod <pod></code> , lower requests or give Minikube more resources
<code>Running but 0/1</code>	Readiness probe failing	<code>kubectl describe pod <pod></code> , look at the Events

```
kubectl get deployment,replicaset,service,endpoints
```

- **What:** Shows the Deployment, the ReplicaSet it created, the Service, and the Service's endpoints (the pod IPs it currently sends traffic to).
- **Expected:** Deployment `3/3`, Service `NodePort 80:30080/TCP`, endpoints listing three `IP:5000` addresses.

- **If it fails:** Empty endpoints (`<none>`) means the Service selector matches no ready pods. Compare the labels with `kubectl get pods --show-labels`.

Open the app

```
minikube service flask-app-service --url
```

- **What:** Opens a tunnel from your laptop to the NodePort and prints the URL.
- **Why:** With the Docker driver on Windows and macOS, the Minikube node IP is not directly reachable, so a tunnel is needed.
- **Where:** Git Bash. **Keep this terminal open;** closing it closes the tunnel.
- **Expected:** `http://127.0.0.1:<random-port>`. Open it in the browser.
- **If it fails:** If the page does not load, check the endpoints (above) and that the pods are `1/1`.

```
for i in $(seq 1 6); do curl -s <URL>/health; echo; done
```

- **What:** Calls the Service six times.
- **Expected:** The `pod` field changes between the three pod names: that is the Service load balancing. (A browser reuses one connection, so it often keeps hitting the same pod.)

Phase 6 - Replicas, Probes and Resource Limits

Replicas

The Deployment creates a **ReplicaSet**, whose controller constantly compares "desired = 3" with "actual = number of running pods with the label" and creates or deletes pods to close the gap. This loop is called **reconciliation** and it is the heart of Kubernetes.

```
kubectl scale deployment flask-app --replicas=5
kubectl get pods -l app=flask-app
```

- **What:** Changes the desired replica count to 5.
- **Expected:** Two new pods appear. Run `kubectl scale ... --replicas=3` to go back; two pods move to `Terminating`.
- **Note:** `kubectl scale` changes the live object only. The YAML in Git still says 3, and the next `kubectl apply` resets it. The YAML file is the source of truth.

Readiness vs liveness probes

	Readiness probe	Liveness probe
Question	Can this pod receive traffic right now?	Is this container stuck or broken?
On failure	Pod is removed from the Service endpoints (no restart)	kubelet kills and restarts the container
Typical use	Startup warm-up, temporary overload, dependency down	Deadlock, hung process
Our settings	every 5s, 3 failures = not ready (about 15s)	starts after 10s, every 10s, 3 failures = restart (about 30s)

Both use the same `httpGet` check, defined once with a YAML anchor (`&health-check`) and reused (`*health-check`). Liveness is deliberately slower and more tolerant than readiness, because a restart is more disruptive than a short removal from traffic.

There is also a `startupProbe` for slow-starting apps. It is not needed here because Flask starts in under a second.

Requests vs limits

	Requests	Limits
Meaning	Guaranteed minimum; used by the scheduler to place the pod	Hard maximum enforced at runtime
CPU above it	-	Throttled (slowed down), never killed
Memory above it	-	Container is killed: <code>OOMKilled</code>
Ours	100m CPU, 64Mi	250m CPU, 128Mi

`100m` = 100 millicores = 0.1 CPU core. `Mi` = mebibytes. Because requests are lower than limits, the pods are in the **Burstable** QoS class (requests = limits would be **Guaranteed**).

```
kubectl top pods -l app=flask-app
kubectl describe pod -l app=flask-app | grep -A3 -E "Limits|Requests|Liveness|Readiness|QoS"
```

- **What:** Shows current usage, then the configured resources, probes and QoS class.
- **Expected:** Each pod uses about 1m CPU and 30-40Mi memory while idle. **QoS Class: Burstable.**
- **If it fails:** `Metrics API not available` means metrics-server is still starting. Wait a minute.

Rolling updates and rollback

```
kubectl rollout restart deployment/flask-app
kubectl rollout status deployment/flask-app
kubectl rollout history deployment/flask-app
```

- **What:** Replaces the pods one at a time, waits for the rollout to finish, then lists the revisions.
- **Why:** With `maxUnavailable: 0` and `maxSurge: 1`, Kubernetes starts a new pod, waits until its readiness probe passes, and only then removes an old one. Users see no downtime.
- **Expected:** `deployment "flask-app" successfully rolled out.`
- `kubectl rollout undo deployment/flask-app` returns to the previous revision.

Phase 7 - Prometheus and Grafana

How Prometheus collects metrics

- **Pull model:** Prometheus calls HTTP endpoints (targets) on a schedule (`scrape_interval: 15s`) and stores each value with a timestamp. Applications do not push anything.
- **Exposition format:** A target returns plain text such as `flask_http_requests_total{endpoint="/health",method="GET",status="200"} 42.`
- **Time series:** Metric name + set of labels = one series. `sum by (pod)` and similar operations group series by their labels.
- **Service discovery:** Instead of a fixed list of IPs, Prometheus asks the Kubernetes API for all pods and nodes (`kubernetes_sd_configs`). Relabeling rules filter and rename them. New replicas are found automatically.
- **RBAC:** Asking the API server for pods and nodes needs permission, which is what the ServiceAccount, ClusterRole and ClusterRoleBinding grant (read-only).

Our three scrape jobs, in `monitoring/10-prometheus.yaml`:

Job	Target	Gives us
<code>prometheus</code>	Prometheus itself	Proves scraping works
<code>kubernetes-cadvisor</code>	kubelet's built-in cAdvisor, via the API server proxy	<code>container_cpu_usage_seconds_total</code> , <code>container_memory_working_set_bytes</code> for every container
<code>app-pods</code>	Every pod annotated <code>prometheus.io/scrape: "true"</code> , on its port named <code>http</code>	Request counts and latency from <code>/metrics</code>

Why plain YAML and not Helm

Helm (and the kube-prometheus-stack chart) installs dozens of objects you would not be able to explain yet. Our version has one Deployment, one Service and one ConfigMap per tool, plus RBAC, and every line is commented. Helm is the natural next step once these basics are clear.

Install

```
kubectl apply -f monitoring/
kubectl create configmap grafana-dashboards -n monitoring \
  --from-file=monitoring/dashboards/ --dry-run=client -o yaml | kubectl apply -f -
```

- **What:** Creates the `monitoring` namespace, Prometheus (RBAC, config, Deployment, Service) and Grafana. The second command turns the dashboard JSON file into a ConfigMap. `--dry-run=client -o yaml | kubectl apply -f -` means "generate the YAML, then apply it", so re-running updates it instead of failing with "already exists".
- **Why:** The files are numbered (`00-`, `10-`, `20-`) because `kubectl apply -f <folder>` processes them alphabetically, and the namespace must exist first.
- **Where:** Git Bash, project root.
- **Expected:** `namespace/monitoring created`, `clusterrole.../prometheus created`, ... `configmap/grafana-dashboards created`.
- **If it fails:** `namespaces "monitoring" not found` means the files were applied out of order; run the command again. A Grafana pod stuck in `ContainerCreating` with a `FailedMount` event means the `grafana-dashboards` ConfigMap is missing; run the second command.

```
kubectl get pods -n monitoring
```

- **Expected:** `prometheus-...` and `grafana-...` both `1/1 Running` within about a minute (the images are downloaded the first time).

Check the targets

```
minikube service prometheus -n monitoring --url
```

Open the URL, then **Status > Targets**.

- **Expected:** `prometheus (1/1 up)`, `kubernetes-cadvisor (1/1 up)`, `app-pods (3/3 up)`.
- **If it fails:** If `kubernetes-cadvisor` shows `403 Forbidden`, the ClusterRole is missing `nodes/proxy` or the binding names the wrong ServiceAccount. If `app-pods` shows 0 targets, check that the pod template has the annotation and that the port is named `http`.

PromQL starter pack

Try these on the Prometheus **Graph** page:

Query	Meaning
<code>up</code>	1 = target scraped OK, 0 = down
<code>flask_http_requests_total</code>	Raw counters (always increasing)
<code>rate(flask_http_requests_total[1m])</code>	Per-second increase over the last minute
<code>sum by (pod) (rate(flask_http_requests_total[1m]))</code>	Requests per second per pod
<code>rate(container_cpu_usage_seconds_total{container="flask-app"}[1m])</code>	CPU cores used per container
<code>container_memory_working_set_bytes{container="flask-app"}</code>	Memory per container (what the OOM killer looks at)

Rule: use `rate()` on counters (names ending in `_total` or `_seconds_total`); use gauges (memory) directly.

Phase 8 - The Grafana Dashboard

Open Grafana

```
minikube service grafana -n monitoring --url
```

- **Where:** A separate Git Bash window; keep it open.
- **Expected:** A URL. Log in with `admin / admin`; Grafana then asks you to choose a new password.
- **If it fails:** If the page does not load, check `kubectl get pods -n monitoring`. A Grafana pod that is `0/1` is still starting.

Go to **Dashboards > Flask App on Kubernetes**. The data source and dashboard were **provisioned**: Grafana read them from the files mounted by the ConfigMaps at startup, so nothing has to be clicked by hand, and a reinstall brings them back.

What each panel shows

Panel	PromQL	Source
Healthy pods being scraped	<code>sum(up{job="app-pods"})</code>	Prometheus
Total requests / second	<code>sum(rate(flask_http_requests_total[...]))</code>	App /metrics
CPU usage per pod	<code>sum by (pod) (rate(container_cpu_usage_seconds_total{container="flask-app"}[...]))</code>	cAdvisor
Memory usage per pod	<code>sum by (pod) (container_memory_working_set_bytes{container="flask-app"})</code>	cAdvisor
Requests per second by pod	<code>sum by (pod) (rate(flask_http_requests_total[...]))</code>	App /metrics
95th percentile response time	<code>histogram_quantile(0.95, sum by (le, endpoint) (rate(..._bucket[...])))</code>	App /metrics

`$__rate_interval` is a Grafana variable that chooses a safe `rate()` window based on the scrape interval and zoom level.

Build one panel yourself (practice)

1. **Dashboards > New > New dashboard > Add visualization**, and choose **Prometheus**.

2. Query: `sum by (pod) (container_memory_working_set_bytes{container="flask-app"})`.
3. Legend: `{{pod}}`. Unit (right panel, Standard options): **bytes (IEC)**.
4. **Apply**, then save. **Share > Export > Save to file** exports the JSON. This is how `flask-app.json` was made.

Generate traffic

```
./scripts/load.sh 50
```

- **What:** Starts a temporary `busybox` pod inside the cluster that calls `http://flask-app-service/work?ms=50` in a loop. An optional second argument sets the number of parallel clients.
- **Why:** Idle graphs are flat. Load makes CPU, requests per second and latency move, which makes for good screenshots. Because traffic goes through the Service, the three pods share it.
- **Where:** Git Bash, project root. Stop it with `Ctrl+C` (the pod is deleted automatically because of `--rm`).
- **Expected:** Sending load with 1 client(s), 50 ms per request. After 30 seconds the dashboard shows activity.
- **If it fails:** `pods "load-generator" already exists` means a previous run is still there; run `kubectl delete pod load-generator`. Unable to use a TTY in Git Bash means winpty is missing; run it from PowerShell instead.

Screenshots for the README

Save them to `docs/screenshots/` with exactly these names (the README already links them):

- `grafana-dashboard.png` - the whole dashboard under load
- `prometheus-targets.png` - Status > Targets with all jobs up
- `Pods-running.png` - terminal showing `kubectl get pods -A`
- `self-healing.png` - terminal during the pod-deletion test (Phase 9)

Phase 9 - Testing and Troubleshooting

Open the Grafana dashboard and a terminal side by side for these experiments. The general troubleshooting method and the error catalogue are in the **Troubleshooting** chapter.

Test 1 - Self-healing (pod deletion)

```
kubectl get pods -l app=flask-app -w
```

In a second terminal:

```
kubectl delete pod <one-flask-app-pod-name>
```

- **What:** `-w` (watch) streams changes. Deleting a pod removes one replica.
- **Expected:** The deleted pod goes `Terminating`, and **immediately** a new pod with a new name appears: `Pending`, then `ContainerCreating`, then `Running 0/1`, then `1/1` once readiness passes. The "Healthy pods" panel briefly drops to 2.
- **Why it happens:** The ReplicaSet saw actual (2) is less than desired (3) and created a pod. Pods are disposable ("cattle, not pets"); the Deployment is what lasts.

- Try `kubectl delete pods -l app=flask-app` (all three at once) and watch all of them come back.

Test 2 - Liveness probe restarts a broken container

```
kubectl port-forward deployment/flask-app 8080:5000
```

In a second terminal:

```
curl -X POST localhost:8080/fail
kubectl get pods -l app=flask-app -w
```

- **What:** `port-forward` connects to **one** pod (bypassing the Service). `/fail` makes that pod's `/health` return 503.
- **Expected:** After about 15s the pod shows `0/1` (readiness failed, so it is removed from the Service). After about 30s the **RESTARTS** column goes to 1 (liveness failed, so the container restarted) and the pod becomes `1/1` again, because a fresh process starts healthy.
- `kubectl describe pod <name>` shows Events such as **Readiness probe failed: HTTP probe failed with statuscode: 503** and **Container flask-app failed liveness probe, will be restarted**.
- **Lesson:** Readiness protects users (no traffic to a bad pod); liveness repairs the pod.

Test 3 - A bad release does not take the site down

```
kubectl set image deployment/flask-app flask-app=flask-app:9.9.9
kubectl get pods -l app=flask-app
kubectl rollout status deployment/flask-app --timeout=30s
```

- **What:** Updates the Deployment to an image tag that does not exist.
- **Expected:** One new pod is stuck in `ErrImagePull` / `ImagePullBackOff`, but the **three old pods keep running**: with `maxUnavailable: 0`, Kubernetes never removes an old pod until a new one is ready. `rollout status` times out.
- **Fix:**

```
kubectl rollout undo deployment/flask-app
```

- **Expected:** `deployment.apps/flask-app` **rolled back**. The broken pod disappears.

Test 4 - Memory limit and OOMKilled

```
kubectl set resources deployment flask-app --requests=memory=16Mi --limits=memory=16Mi
kubectl get pods -l app=flask-app -w
```

- **What:** Lowers the memory limit below what Python and gunicorn need. The request is lowered too, because a request may never be larger than its limit.
- **Expected:** New pods go to `OOMKilled`, then `CrashLoopBackOff`. `kubectl describe pod <name>` shows **Last State: Terminated, Reason: OOMKilled, Exit Code: 137** (137 = 128 + signal 9, SIGKILL). Old pods keep serving traffic, as in Test 3.
- **Fix:** `kubectl rollout undo deployment/flask-app`, or `kubectl apply -f k8s/` to return to the version in Git.

Test 5 - CPU limit and throttling

```
./scripts/load.sh 500 6
```

- **What:** 6 parallel clients, each asking for 500 ms of CPU work per request, which is far more than 3 pods x 0.25 CPU can deliver.
- **Expected:** In Grafana, each pod's CPU line flattens out at the CPU limit and does not go higher. Latency (p95) rises. Nothing restarts, because CPU over the limit is throttled, never killed. This shows the difference between CPU and memory limits.

Test 6 - Scaling under load

While `load.sh` runs, scale to 5 replicas and watch the "Requests per second by pod" panel: the new pods start receiving traffic as soon as they are ready, and the per-pod CPU drops.

Clean up

```
./scripts/teardown.sh  
minikube stop
```

- **What:** Deletes the app and monitoring objects, then stops (but keeps) the cluster. `minikube start` brings it back. `minikube delete` removes it completely.

Phase 10 - GitHub Repository and README

What goes into Git

Code, Dockerfile, YAML, scripts, docs and screenshots. **Not** `.venv/`, caches or secrets; `.gitignore` handles this. `.gitattributes` forces Linux line endings so shell scripts and YAML work inside Linux containers even when edited on Windows.

Commit and push

```
git add .  
git commit -m "Describe what changed and why"  
git remote add origin https://github.com/<your-username>/k8s-flask-monitoring.git  
git push -u origin main
```

- **What:** Stages all changes, records a commit, links the local repo to the GitHub repo, and uploads the `main` branch (`-u` remembers the link, so later you only type `git push`).
- **Where:** Git Bash, project root. First create an **empty** repository on github.com (no README, no `.gitignore`) with the same name.
- **Expected:** `branch 'main' set up to track 'origin/main'`.
- **If it fails:** `remote origin already exists` means you should run `git remote set-url origin <url>`. `Authentication failed` happens because GitHub no longer accepts passwords; sign in through the browser window that Git Credential Manager opens, or use a personal access token. `rejected (fetch first)` means the GitHub repo was not empty; run `git pull --rebase origin main` and push again. If commits show someone else's name, the email in `git config user.email` is not on your GitHub account.

Make the repository look professional

- Add a description and topics on GitHub: `kubernetes`, `docker`, `prometheus`, `grafana`, `minikube`, `flask`, `devops`.

- Add the four screenshots from Phase 8 to [docs/screenshots/](#) and push them.
- Pin the repository on your GitHub profile.
- Write good commit messages: a short summary line in the imperative mood ("Add readiness probe"), then a blank line, then why.

Troubleshooting

The method: get, describe, logs, events, exec

Almost every Kubernetes problem is solved by running these five commands in order, from the broadest view to the most detailed.

Step	Command	Answers
1. Get	<code>kubectl get pods -A</code>	Which pod is unhappy, and what is its STATUS / RESTARTS?
2. Describe	<code>kubectl describe pod <name></code>	Events at the bottom: scheduling, image pulls, probe failures, OOM kills
3. Logs	<code>kubectl logs <name></code> (add <code>--previous</code> after a crash)	What the application itself printed
4. Events	<code>kubectl get events --sort-by=.lastTimestamp</code>	Cluster-wide timeline of what just happened
5. Exec	<code>kubectl exec -it <name> -- sh</code>	Look inside the running container (files, env, network)

Two more checks solve most networking problems:

Check	Command	Healthy result
Does the Service have pods behind it?	<code>kubectl get endpoints flask-app-service</code>	Three <code>IP:5000</code> entries
Do the labels match?	<code>kubectl get pods --show-labels</code>	Pods carry <code>app=flask-app</code>

Error catalogue

Symptom	Likely cause	How to confirm	Fix
<code>ErrImagePull</code> / <code>ImagePullBackOff</code>	Image not inside Minikube, tag typo, or <code>imagePullPolicy: Always</code>	<code>describe</code> Events: pull access denied / not found	<code>minikube image load flask-app:1.0.0</code> ; make the tag match <code>k8s/deployment.yaml</code>
<code>CrashLoopBackOff</code>	App exits on start (bad code, bad config, OOM)	<code>kubectl logs <pod> --previous</code>	Fix the error in the logs; for exit code 137 see <code>OOMKilled</code>
<code>OOMKilled</code> (exit code 137)	Memory above the limit	<code>describe: Last State: Terminated, Reason: OOMKilled</code>	Raise <code>limits.memory</code> or fix the memory leak
<code>Pending</code>	Not enough CPU or memory for the requests on any node	<code>describe: 0/1 nodes are available: Insufficient cpu</code>	Lower requests, scale down, or <code>minikube start --cpus/--memory</code>
<code>CreateContainerConfigError</code>	Referenced ConfigMap or Secret does not exist	<code>describe: configmap "..."</code> not found	<code>kubectl apply -f k8s/configmap.yaml</code>
<code>ContainerCreating</code> forever	A volume cannot be mounted	<code>describe: FailedMount</code>	Create the missing ConfigMap (e.g. <code>grafana-dashboards</code>)
<code>Running</code> but <code>READY 0/1</code>	Readiness probe failing	<code>describe: Readiness probe failed: ... 503</code>	Check <code>/health</code> with <code>kubectl port-forward</code> ; check the probe path and port
<code>RESTARTS</code> keeps increasing	Liveness probe failing, or crashes	<code>describe: failed liveness probe, will be restarted</code>	Make sure the app answers <code>/health</code> in time; increase <code>initialDelaySeconds</code> for slow starts
Browser cannot reach the app	Tunnel closed, or Service has no endpoints	<code>kubectl get endpoints flask-app-service</code>	Keep <code>minikube service ...</code> running; fix the selector/labels
<code>connection refused</code> from <code>kubectl</code>	Cluster stopped or wrong context	<code>kubectl config current-context</code>	<code>minikube start</code> ; <code>kubectl config use-context minikube</code>
Rollout stuck	New pods never become ready	<code>kubectl rollout status deployment/flask-app</code>	<code>kubectl rollout undo deployment/flask-app</code> , then investigate the new pod
Prometheus target <code>DOWN</code>	Pod not ready, wrong port, or <code>/metrics</code> missing	Status > Targets shows the error text	Port must be named <code>http</code> ; check <code>curl <pod>:5000/metrics</code>
Prometheus cAdvisor <code>403</code>	RBAC missing <code>nodes/proxy</code>	Status > Targets	Re-apply <code>monitoring/10-prometheus.yaml</code>
Grafana panel <code>No data</code>	Wrong query, wrong time range, or no traffic yet	Run the query in Prometheus > Graph	Generate load with <code>scripts/load.sh</code> ; set the time range to the last 15 minutes
<code>minikube start</code> fails	Docker Desktop not running or short of memory	Error message mentions the driver or memory	Start Docker Desktop; lower <code>--memory</code> ; <code>minikube delete</code> and retry
Shell script fails with <code>\$'\r': command not found</code>	Windows (CRLF) line endings	file <code>scripts/deploy.sh</code>	<code>.gitattributes</code> prevents it; fix existing files with <code>git add --renormalize .</code>

Useful one-liners

```
kubectl get all -n monitoring          # everything in a namespace
kubectl logs -l app=flask-app --tail=20 --prefix  # last logs from every replica
kubectl top pods -A                   # live CPU/memory (metrics-server)
kubectl get pod <name> -o yaml        # the full live object
kubectl explain deployment.spec.strategy  # built-in documentation for any field
minikube dashboard                   # Kubernetes web UI
```

Interview Preparation

The 60-second project pitch

"I built a small Flask web app and focused on the DevOps around it. I containerised it with a slim, non-root Docker image and deployed it to a local Kubernetes cluster on Minikube. A Deployment runs three replicas with readiness and liveness probes and CPU and memory limits, and a NodePort Service load-balances across them. For monitoring, I set up Prometheus and Grafana with plain YAML: Prometheus discovers pods through the Kubernetes API and scrapes container CPU and memory from cAdvisor plus request metrics from the app, and Grafana shows them on a provisioned dashboard. Then I tested failure: deleting pods, making the health check fail, deploying a broken image and setting a memory limit too low. I used describe, logs and events to diagnose each one, and rollbacks to recover."

Core questions

Why is Kubernetes used?

Running one container is easy; running many reliably is hard. Kubernetes is a container orchestrator. You declare the desired state ("3 replicas of this image with these limits") and its controllers keep making the real state match: they restart crashed containers, replace lost pods, schedule pods onto nodes with free capacity, load-balance traffic through Services, roll out new versions without downtime and roll them back. Doing all of that by hand with `docker run` does not scale.

What is the difference between a Pod, a Deployment and a Service?

- **Pod:** the smallest deployable unit, one or more containers sharing an IP and storage. Pods are temporary: when one dies it is replaced by a new pod with a new name and IP, never repaired.
- **Deployment:** manages pods through a ReplicaSet. It keeps the replica count, and handles rolling updates and rollbacks. You almost never create bare pods.
- **Service:** a stable virtual IP and DNS name (`flask-app-service`) that load-balances to all **ready** pods matching its label selector. It solves the problem that pod IPs keep changing.

In my project the Deployment creates 3 pods labelled `app: flask-app`, and the Service selects that label.

How do replicas work?

`replicas: 3` is the desired state. The Deployment creates a ReplicaSet whose controller runs a reconciliation loop: it counts the pods matching its selector and creates or deletes pods until the count is 3. Scaling is just changing that number (`kubectl scale` or editing the YAML). Replicas give availability (one pod can die without an outage) and capacity (the Service spreads requests across them, as my Grafana "requests by pod" panel shows).

How does Kubernetes handle pod failure?

There are two levels:

1. **Container crash or failed liveness probe:** the kubelet on the node restarts the container inside the same pod (`RESTARTS` goes up). Repeated crashes lead to `CrashLoopBackOff`, with growing delays between restarts.
2. **Pod deleted, evicted, or its node lost:** the ReplicaSet sees fewer pods than desired and creates a replacement pod, possibly on another node.

Meanwhile the readiness probe removes unhealthy pods from the Service, so users are not sent to them. I demonstrated this by deleting a pod and watching a new one appear within seconds.

What is the difference between liveness and readiness probes?

- **Readiness:** "Can this pod take traffic now?" If it fails, the pod is removed from the Service endpoints but **not** restarted. Useful during startup or temporary overload.
- **Liveness:** "Is this container broken beyond self-recovery?" If it fails, the container is **restarted**. Useful for deadlocks or hung processes.

Both of mine call `GET /health`. The liveness probe is slower and more tolerant (restart after about 30s) than readiness (removed after about 15s), because a restart is the bigger action. A common mistake is a liveness probe that checks a database: if the database goes down, every pod restarts in a loop, which makes things worse.

How does Prometheus collect metrics?

It **pulls**: every 15 seconds it sends an HTTP GET to each target's metrics endpoint and stores the values as time series (metric name + labels + timestamp + value). It finds targets through **Kubernetes service discovery**, asking the API server for pods and nodes (which is why it needs RBAC permissions), and relabeling rules choose which ones to keep. I scrape three things: Prometheus itself, cAdvisor in the kubelet for container CPU and memory, and my app's `/metrics` endpoint, which uses the `prometheus_client` library to expose a request counter and a latency histogram.

Pull model benefits: Prometheus knows when a target is down (`up == 0`), and apps do not need to know where Prometheus is.

How does Grafana visualise monitoring data?

Grafana stores no metrics itself. It connects to Prometheus as a **data source**, sends **PromQL** queries for the selected time range, and draws the results as panels (time series, stats). My data source and dashboard are **provisioned** from files in ConfigMaps, so they are recreated automatically and version-controlled in Git, rather than clicked together by hand.

How do you troubleshoot a failed deployment?

I follow the same order every time:

1. `kubectl get pods`: read the STATUS (`ImagePullBackOff`, `CrashLoopBackOff`, `Pending`, `0/1`).
2. `kubectl describe pod <name>`: the Events usually say exactly what failed (image not found, insufficient CPU, probe returned 503, OOMKilled).
3. `kubectl logs <name> --previous`: the application's own error if it crashed.
4. `kubectl get endpoints <service>` if the app runs but cannot be reached: empty endpoints mean a label/selector mismatch or no ready pods.
5. Fix the YAML and `kubectl apply`, or `kubectl rollout undo` to recover immediately and investigate afterwards.

Example from my project: I deployed a non-existent image tag. The new pod went to `ImagePullBackOff`, but because of `maxUnavailable: 0` the old pods kept serving. `rollout undo` fixed it in seconds.

Follow-up questions to be ready for

Question	Short answer
Requests vs limits?	Requests are reserved and used for scheduling; limits are the maximum. CPU over the limit is throttled; memory over the limit is OOMKilled.
What is OOMKilled / exit code 137?	The kernel killed the process for exceeding its memory limit (137 = 128 + SIGKILL 9).
ClusterIP vs NodePort vs LoadBalancer?	Internal only / also on each node's port 30000-32767 / cloud load balancer in front.
Why a ConfigMap?	Configuration changes without rebuilding the image; the same image moves from dev to prod. Secrets are for sensitive values.
What is a namespace?	A logical partition of a cluster. I separate monitoring from the app.
How does a rolling update work?	New ReplicaSet scales up while the old one scales down, gated by readiness and maxSurge/maxUnavailable .
Why rate() in PromQL?	Counters only increase; rate() turns them into per-second change.
Why not Helm?	To learn what each object does first. Helm would be the next step to package and version these manifests.
Why only one gunicorn worker?	Scale with pods, not processes; keeps metrics in one process and resource limits predictable.
Why run as non-root?	If the app is compromised, the attacker has fewer privileges in the container.
What would you add next?	CI pipeline (GitHub Actions) to test and build the image, a HorizontalPodAutoscaler, Alertmanager alerts, Helm or Kustomize, an Ingress.

Resume-ready description

Kubernetes-Based Application Deployment and Monitoring | Docker, Kubernetes (Minikube), Prometheus, Grafana, Python Flask, Linux, Git

- Containerised a Python Flask web application with a slim, non-root Docker image using layer caching, and deployed it to Kubernetes (Minikube) with Deployment, Service and ConfigMap manifests.
- Configured 3 replicas with rolling updates (zero unavailable pods), readiness and liveness probes, and CPU/memory requests and limits.
- Set up Prometheus with Kubernetes service discovery and RBAC to scrape container CPU/memory (cAdvisor) and custom application metrics, and visualised them on a provisioned Grafana dashboard.
- Validated self-healing and failure handling by deleting pods, failing health checks, deploying a broken image and triggering OOMKills; diagnosed each with `kubectl describe`, logs and events and recovered with rollbacks.
- Documented architecture, deployment and troubleshooting steps, and added automated tests that validate the application and the consistency of the Kubernetes manifests.

Appendix - Source Files

Every file below is included directly from the repository, so it always matches the code you run. The guide chapters explain why each part exists.

app/config.py

```
"""Single source of truth for application settings (DRY).

Every value has a sensible default and can be overridden with an environment
variable, so the SAME image runs unchanged on a laptop, in Docker and in
Kubernetes. Nothing else in the code base hardcodes these values.
"""

import os
from dataclasses import dataclass, field

def _env(name: str, default: str) -> str:
    return os.getenv(name, default)

@dataclass(frozen=True)
class Settings:
    app_name: str = field(default_factory=lambda: _env("APP_NAME", "flask-app"))
    app_version: str = field(default_factory=lambda: _env("APP_VERSION", "dev"))
    app_env: str = field(default_factory=lambda: _env("APP_ENV", "local"))
    port: int = field(default_factory=lambda: int(_env("PORT", "5000")))
    # Inside Kubernetes HOSTNAME is the pod name, which shows which replica answered.
    hostname: str = field(default_factory=lambda: _env("HOSTNAME", "localhost"))

settings = Settings()
```

app/app.py

```
"""A deliberately small Flask app: the focus of this project is DevOps, not features.

Endpoints
GET /           homepage (shows which pod answered)
GET /health     used by Kubernetes readiness and liveness probes
GET /metrics    Prometheus scrape endpoint
GET /work       burns a little CPU so the Grafana graphs move (?ms=100)
POST /fail      makes /health return 503 so you can watch the liveness probe restart the po
d
"""
import time

from flask import Flask, Response, g, jsonify, request
from prometheus_client import CONTENT_TYPE_LATEST, Counter, Histogram, generate_latest

from config import settings

app = Flask(__name__)

REQUESTS = Counter(
    "flask_http_requests_total", "Total HTTP requests", ["method", "endpoint", "status"]
)
LATENCY = Histogram(
    "flask_http_request_duration_seconds", "HTTP request latency in seconds", ["endpoint"]
)

MAX_WORK_MS = 1000

# Per-process flag. Flipped by POST /fail to simulate an app that is "running but broken".
_state = {"healthy": True}

HOMEPAGE = """<!doctype html>
<html><head><title>{name}</title>
<style>body{{font-family:sans-serif;max-width:640px;margin:60px auto;line-height:1.6}}
code{{background:#f3f3f3;padding:2px 6px;border-radius:4px}}</style></head>
<body>
<h1>Hello from {name} &#128075;</h1>
<p>Served by pod <code>{hostname}</code></p>
<p>Version <code>{version}</code> &middot; environment <code>{env}</code></p>
<p>Refresh the page: when the request goes through the Kubernetes Service, a different pod m
ay answer.</p>
<p>Try <a href="/health">/health</a> and <a href="/metrics">/metrics</a>.</p>
</body></html>"""

@app.before_request
def _start_timer():
    g.start = time.perf_counter()

@app.after_request
def _record_metrics(response):
    # Use the route pattern (e.g. "/work"), not the raw URL, so query strings or
    # random paths cannot create unlimited label values (high cardinality).
    endpoint = request.url_rule.rule if request.url_rule else "unmatched"
    if endpoint != "/metrics":
        REQUESTS.labels(request.method, endpoint, response.status_code).inc()
        LATENCY.labels(endpoint).observe(time.perf_counter() - g.start)
    return response
```

```

@app.get("/")
def home():
    return HOMEPAGE.format(
        name=settings.app_name,
        hostname=settings.hostname,
        version=settings.app_version,
        env=settings.app_env,
    )

@app.get("/health")
def health():
    if _state["healthy"]:
        return jsonify(status="ok", pod=settings.hostname)
    return jsonify(status="unhealthy", pod=settings.hostname), 503

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

@app.get("/work")
def work():
    ms = min(request.args.get("ms", default=100, type=int), MAX_WORK_MS)
    deadline = time.perf_counter() + ms / 1000
    count = 0
    while time.perf_counter() < deadline:
        count += 1
    return jsonify(busy_ms=ms, iterations=count, pod=settings.hostname)

@app.post("/fail")
def fail():
    _state["healthy"] = False
    return jsonify(status="health check will now fail", pod=settings.hostname)

if __name__ == "__main__":
    # Local development only. In the container gunicorn runs the app (see gunicorn.conf.py).
    app.run(host="0.0.0.0", port=settings.port, debug=True)

```

app/gunicorn.conf.py

```

"""Production web server settings. The port comes from config.py (DRY)."""
from config import settings

bind = f"0.0.0.0:{settings.port}"

# One process per container: in Kubernetes we scale by adding pods (replicas),
# not workers. It also keeps the Prometheus counters in a single process.
workers = 1
threads = 4

accesslog = "-" # log requests to stdout so `kubectl logs` shows them

```

app/requirements.txt

```

Flask==3.1.0
prometheus-client==0.21.1
gunicorn==23.0.0

```

Dockerfile

```
# Small official Python base image ("slim" = Debian without extras).
FROM python:3.12-slim

# Version is passed in at build time (scripts/deploy.sh reads it from k8s/deployment.yaml).
ARG APP_VERSION=dev

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_VERSION=${APP_VERSION}

WORKDIR /app

# Copy requirements first: this layer is cached and only rebuilt when dependencies change.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

# Never run as root inside a container.
RUN useradd --uid 10001 --no-create-home appuser
USER 10001

# Documentation only; the real port comes from config.py (PORT, default 5000).
EXPOSE 5000

CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:app"]
```

.dockerignore

```
# Only app/ is needed in the image. Excluding the rest keeps the build context small.
.git
.venv
**/__pycache__
.pytest_cache
tests
docs
k8s
monitoring
scripts
*.md
*.pdf
```

k8s/configmap.yaml

```
# Non-secret configuration, kept outside the image so it can change without a rebuild.
apiVersion: v1
kind: ConfigMap
metadata:
  name: flask-app-config
  labels:
    app: flask-app
data:
  APP_ENV: minikube
```

k8s/deployment.yaml

```
# Deployment = "keep N identical pods of this image running, and replace any that die".
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  labels:
    app: flask-app
spec:
  replicas: 3
  # The Deployment owns every pod whose labels match this selector.
  selector:
    matchLabels:
      app: flask-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # never drop below 3 ready pods during an update
      maxSurge: 1 # create at most 1 extra pod while updating
  # Template = the blueprint for each pod.
  template:
    metadata:
      labels:
        app: flask-app # must match spec.selector above
      annotations:
        prometheus.io/scrape: "true" # tells our Prometheus config to scrape this pod
    spec:
      securityContext:
        runAsNonRoot: true
      containers:
        - name: flask-app
          # Single source of truth for the image tag: scripts/deploy.sh reads it from here.
          image: flask-app:1.0.0
          imagePullPolicy: IfNotPresent # use the image loaded into Minikube; don't pull from Docker Hub
          ports:
            - name: http # everything below refers to the port by this name, not the number
              containerPort: 5000
          envFrom:
            - configMapRef:
                name: flask-app-config
          resources:
            requests: # what the scheduler reserves for the pod
              cpu: 100m # 0.1 CPU core
              memory: 64Mi
            limits: # hard ceiling: CPU is throttled, memory over the limit = OOMKilled
              cpu: 250m
              memory: 128Mi
          # Readiness: "can this pod receive traffic right now?" Fails -> removed from the Service.
          readinessProbe:
            httpGet: &health-check # YAML anchor: define the check once...
              path: /health
              port: http
            initialDelaySeconds: 3
            periodSeconds: 5
            failureThreshold: 3
          # Liveness: "is this container still alive?" Fails -> kubelet restarts the container.
          livenessProbe:
            httpGet: *health-check # ...and reuse it here (DRY)

            initialDelaySeconds: 10
            periodSeconds: 10
            failureThreshold: 3
```

k8s/service.yaml

```
# Service = one stable name/IP in front of the changing set of pods, with load balancing.
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
  labels:
    app: flask-app
spec:
  type: NodePort # reachable from outside the cluster on <node-ip>:30080
  selector:
    app: flask-app # send traffic to every READY pod with this label
  ports:
    - name: http
      port: 80 # port inside the cluster: http://flask-app-service
      targetPort: http # the container's named port (5000)
      nodePort: 30080 # port on the Minikube node (allowed range 30000-32767)
```

monitoring/00-namespace.yaml

```
# Keep monitoring tools separate from the application.
# Files in this folder are numbered so `kubectl apply -f monitoring/` creates the namespace
# first.
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
```

monitoring/10-prometheus.yaml

```
# Prometheus, installed with plain YAML (no Helm) so every line can be read and explained.
#
# 1. ServiceAccount + ClusterRole + ClusterRoleBinding: permission to discover pods/nodes
# 2. ConfigMap: prometheus.yml (WHAT to scrape)
# 3. Deployment: runs the Prometheus server
# 4. Service: exposes the web UI on NodePort 30090
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: monitoring
---
# Read-only access. "nodes/proxy" lets Prometheus read cAdvisor (container CPU/memory) via the API server.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
  - apiGroups: [""]
    resources: [nodes, nodes/proxy, nodes/metrics, services, endpoints, pods]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
  - kind: ServiceAccount
    name: prometheus
    namespace: monitoring
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s # how often Prometheus PULLS metrics from every target

    scrape_configs:
      # Prometheus monitors itself.
      - job_name: prometheus
        static_configs:
          - targets: ["localhost:9090"]

      # Container CPU and memory from cAdvisor (built into the kubelet on every node).
      # We go through the API server proxy so it works the same on any cluster.
      - job_name: kubernetes-cadvisor
        scheme: https
        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        authorization:
```

```

    credentials_file: /var/run/secrets/kubernetes.io/serviceaccount/token
kubernetes_sd_configs:
  - role: node
relabel_configs:
  - target_label: __address__
    replacement: kubernetes.default.svc:443
  - source_labels: [__meta_kubernetes_node_name]
    target_label: __metrics_path__
    replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor

# Application metrics from /metrics on every pod annotated prometheus.io/scrape: "true"
".
# Pods are discovered automatically, so new replicas are scraped without any config change.
- job_name: app-pods
  kubernetes_sd_configs:
    - role: pod
  relabel_configs:
    - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
      action: keep
      regex: "true"
    - source_labels: [__meta_kubernetes_pod_container_port_name]
      action: keep
      regex: http # scrape the port named "http" (no port number duplicated here)
    - source_labels: [__meta_kubernetes_namespace]
      target_label: namespace
    - source_labels: [__meta_kubernetes_pod_name]
      target_label: pod
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: monitoring
  labels:
    app: prometheus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      serviceAccountName: prometheus
      containers:
        - name: prometheus
          image: prom/prometheus:v2.53.0
          args:
            - --config.file=/etc/prometheus/prometheus.yml
            - --storage.tsdb.path=/prometheus
            - --storage.tsdb.retention.time=2d
          ports:
            - name: web
              containerPort: 9090
          readinessProbe:
            httpGet:
              path: /-/ready

```

```

    port: web
  livenessProbe:
    httpGet:
      path: /-/healthy
      port: web
    initialDelaySeconds: 15
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi
  volumeMounts:
    - name: config
      mountPath: /etc/prometheus
    - name: data
      mountPath: /prometheus
  volumes:
    - name: config
      configMap:
        name: prometheus-config
    - name: data
      emptyDir: {} # data is lost when the pod restarts - fine for a learning project
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus
  namespace: monitoring
spec:
  type: NodePort
  selector:
    app: prometheus
  ports:
    - name: web
      port: 9090
      targetPort: web
      nodePort: 30090

```

monitoring/20-grafana.yaml

```
# Grafana, installed with plain YAML (no Helm).
#
# "Provisioning" = Grafana reads config files at startup, so the Prometheus data
# source and our dashboard exist automatically. No clicking needed after a reinstall.
#
# The dashboard itself lives in monitoring/dashboards/flask-app.json (single source).
# scripts/deploy.sh turns it into the "grafana-dashboards" ConfigMap.
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-provisioning
  namespace: monitoring
data:
  datasources.yaml: |
    apiVersion: 1
    datasources:
      - name: Prometheus
        uid: prometheus          # the dashboard JSON refers to this uid
        type: prometheus
        access: proxy
        url: http://prometheus:9090 # Service name -> cluster DNS
        isDefault: true
  dashboards.yaml: |
    apiVersion: 1
    providers:
      - name: default
        folder: ""
        type: file
        options:
          path: /var/lib/grafana/dashboards
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
  namespace: monitoring
  labels:
    app: grafana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      labels:
        app: grafana
    spec:
      containers:
        - name: grafana
          image: grafana/grafana:11.1.0
          ports:
            - name: web
              containerPort: 3000
          readinessProbe:
            httpGet:
              path: /api/health
              port: web
          resources:
```

```

    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi
  volumeMounts:
    - name: provisioning
      mountPath: /etc/grafana/provisioning/datasources/datasources.yaml
      subPath: datasources.yaml
    - name: provisioning
      mountPath: /etc/grafana/provisioning/dashboards/dashboards.yaml
      subPath: dashboards.yaml
    - name: dashboards
      mountPath: /var/lib/grafana/dashboards
  volumes:
    - name: provisioning
      configMap:
        name: grafana-provisioning
    - name: dashboards
      configMap:
        name: grafana-dashboards
---
apiVersion: v1
kind: Service
metadata:
  name: grafana
  namespace: monitoring
spec:
  type: NodePort
  selector:
    app: grafana
  ports:
    - name: web
      port: 3000
      targetPort: web
      nodePort: 30030

```

monitoring/dashboards/flask-app.json

```
{
  "uid": "flask-app",
  "title": "Flask App on Kubernetes",
  "description": "CPU, memory, traffic and health of the flask-app Deployment",
  "tags": ["flask", "kubernetes"],
  "timezone": "browser",
  "schemaVersion": 39,
  "version": 1,
  "refresh": "10s",
  "time": { "from": "now-30m", "to": "now" },
  "panels": [
    {
      "id": 1,
      "type": "stat",
      "title": "Healthy pods being scraped",
      "description": "Number of flask-app pods whose /metrics endpoint Prometheus can reach (up == 1).",
      "gridPos": { "x": 0, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(up{job=\"app-pods\"})" }],
      "fieldConfig": {
        "defaults": {
          "thresholds": {
            "mode": "absolute",
            "steps": [
              { "color": "red", "value": null },
              { "color": "orange", "value": 2 },
              { "color": "green", "value": 3 }
            ]
          }
        }
      },
      "overrides": []
    },
    {
      "options": { "colorMode": "background", "graphMode": "none", "reduceOptions": { "calcs": ["lastNotNull"] } }
    },
    {
      "id": 2,
      "type": "stat",
      "title": "Total requests / second",
      "gridPos": { "x": 6, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(rate(flask_http_requests_total[$__rate_interval]))" }],
      "fieldConfig": { "defaults": { "unit": "reqps", "decimals": 2 }, "overrides": [] },
      "options": { "graphMode": "area", "reduceOptions": { "calcs": ["lastNotNull"] } }
    },
    {
      "id": 3,
      "type": "stat",
      "title": "Total CPU used by flask-app",
      "gridPos": { "x": 12, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(rate(container_cpu_usage_seconds_total{namespace=\"default\", container=\"flask-app\"}[$__rate_interval]))" }],
      "fieldConfig": { "defaults": { "unit": "none", "decimals": 3, "displayName": "CPU core s" }, "overrides": [] },
      "options": { "graphMode": "area", "reduceOptions": { "calcs": ["lastNotNull"] } }
    }
  ]
}
```

```

    "id": 4,
    "type": "stat",
    "title": "Total memory used by flask-app",
    "gridPos": { "x": 18, "y": 0, "w": 6, "h": 5 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [ { "refId": "A", "expr": "sum(container_memory_working_set_bytes{namespace=
\\default\\, container=\\flask-app\\})" } ],
    "fieldConfig": { "defaults": { "unit": "bytes" }, "overrides": [] },
    "options": { "graphMode": "area", "reduceOptions": { "calcs": ["lastNotNull"] } }
  },
  {
    "id": 5,
    "type": "timeseries",
    "title": "CPU usage per pod (cores)",
    "description": "Compare with the CPU limit in k8s/deployment.yaml. A flat line at the
limit means the pod is being throttled.",
    "gridPos": { "x": 0, "y": 5, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (rate(container_cpu_usage_seconds_total{namespace=\\default\\
, container=\\flask-app\\}[\\$__rate_interval]))",
        "legendFormat": "{{pod}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "none", "decimals": 3, "min": 0 }, "overrides":
[] }
  },
  {
    "id": 6,
    "type": "timeseries",
    "title": "Memory usage per pod (working set)",
    "description": "Compare with the memory limit in k8s/deployment.yaml. Going above it g
ets the container OOMKilled.",
    "gridPos": { "x": 12, "y": 5, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (container_memory_working_set_bytes{namespace=\\default\\, c
ontainer=\\flask-app\\})",
        "legendFormat": "{{pod}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "bytes", "min": 0 }, "overrides": [] }
  },
  {
    "id": 7,
    "type": "timeseries",
    "title": "Requests per second by pod",
    "description": "Shows the Service spreading traffic across replicas.",
    "gridPos": { "x": 0, "y": 14, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (rate(flask_http_requests_total[\\$__rate_interval]))",
        "legendFormat": "{{pod}}"
      }
    ]
  }

```

```

    ],
    "fieldConfig": { "defaults": { "unit": "reqps", "min": 0 }, "overrides": [] }
  },
  {
    "id": 8,
    "type": "timeseries",
    "title": "95th percentile response time",
    "gridPos": { "x": 12, "y": 14, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "histogram_quantile(0.95, sum by (le, endpoint) (rate(flask_http_request_uration_seconds_bucket[ $__rate_interval ])))",
        "legendFormat": "{{endpoint}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "s", "min": 0 }, "overrides": [] }
  }
]
}

```

scripts/deploy.sh

```

#!/usr/bin/env bash
# Build the image and deploy the app + monitoring to Minikube in one go.
# Run from Git Bash (Windows), WSL or any Linux/macOS terminal: ./scripts/deploy.sh
#
# DRY: the image name and tag are read from k8s/deployment.yaml, so the version
# is written in exactly one place.
set -euo pipefail
cd "$(dirname "$0")/.."

IMAGE=$(awk ' $1 == "image:" {print $2; exit}' k8s/deployment.yaml)
VERSION=${IMAGE##*:}

step() { printf '\n==> %s\n' "$*"; }

step "Starting Minikube (skipped if it is already running)"
minikube status >/dev/null 2>&1 || minikube start --driver=docker --cpus=2 --memory=4096
minikube addons enable metrics-server >/dev/null # enables `kubectl top`

step "Building $IMAGE"
docker build --build-arg APP_VERSION="$VERSION" -t "$IMAGE" .

step "Loading $IMAGE into Minikube"
minikube image load "$IMAGE"

step "Deploying the application"
kubectl apply -f k8s/

step "Deploying Prometheus and Grafana"
kubectl apply -f monitoring/
# Generate the dashboard ConfigMap from the JSON file (create-or-update).
kubectl create configmap grafana-dashboards -n monitoring \
  --from-file=monitoring/dashboards/ --dry-run=client -o yaml | kubectl apply -f -

step "Waiting for everything to become ready"
kubectl rollout status deployment/flask-app --timeout=120s
kubectl rollout status deployment/prometheus -n monitoring --timeout=180s
kubectl rollout status deployment/grafana -n monitoring --timeout=180s

step "Done. Open the services (each command keeps a tunnel open; use a separate terminal):"
cat <<'EOF'
  minikube service flask-app-service
  minikube service prometheus -n monitoring
  minikube service grafana -n monitoring      # login admin / admin
EOF

```

scripts/load.sh

```
#!/usr/bin/env bash
# Generate traffic from INSIDE the cluster so the Grafana graphs move.
# Requests go through the Service, so you can watch the load spread across pods.
#
# Usage: ./scripts/load.sh [busy-ms-per-request] [parallel-clients]
#   ./scripts/load.sh           light load (50 ms, 1 client)
#   ./scripts/load.sh 500 6     heavy load: pods hit their CPU limit
# Stop with Ctrl+C (the pod is deleted automatically).
set -euo pipefail

MS=${1:-50}
CLIENTS=${2:-1}
# winpty is needed for interactive kubectl in Git Bash on Windows.
TTY=$(command -v winpty >/dev/null 2>&1 && echo winpty || true)

$TTY kubectl run load-generator --rm -it --restart=Never --image=busybox:1.36 -- /bin/sh -c
"
  for i in $(seq $CLIENTS); do
    while true; do wget -q -O /dev/null 'http://flask-app-service/work?ms=$MS'; done &
  done
  echo 'Sending load with $CLIENTS client(s), $MS ms per request. Ctrl+C to stop.'
  wait"
```

scripts/teardown.sh

```
#!/usr/bin/env bash
# Remove everything deploy.sh created (the Minikube cluster itself is kept).
set -euo pipefail
cd "$(dirname "$0")/.."

kubectl delete -f k8s/ --ignore-not-found
kubectl delete -f monitoring/ --ignore-not-found # deleting the namespace also removes grafana-dashboards
echo "Removed. Run 'minikube stop' to pause the cluster or 'minikube delete' to remove it."
```

tests/test_app.py

```
import pytest

import app as app_module

@pytest.fixture
def client():
    app_module._state["healthy"] = True
    app_module.app.config["TESTING"] = True
    with app_module.app.test_client() as client:
        yield client

def test_homepage_shows_app_name(client):
    response = client.get("/")
    assert response.status_code == 200
    assert b"flask-app" in response.data

def test_health_is_ok(client):
    response = client.get("/health")
    assert response.status_code == 200
    assert response.get_json()["status"] == "ok"

def test_fail_makes_health_return_503(client):
    client.post("/fail")
    assert client.get("/health").status_code == 503

def test_metrics_counts_requests(client):
    client.get("/health")
    body = client.get("/metrics").get_data(as_text=True)
    assert 'flask_http_requests_total{endpoint="/health",method="GET",status="200"}' in body

def test_work_is_capped(client):
    response = client.get("/work?ms=999999")
    assert response.get_json()["busy_ms"] == app_module.MAX_WORK_MS
```

tests/test_manifests.py

```
"""Offline checks that the Kubernetes and monitoring files agree with each other.

These catch the classic "selector does not match labels" mistakes without needing a cluster.
"""
import json
from pathlib import Path

import pytest

yaml = pytest.importorskip("yaml")

ROOT = Path(__file__).resolve().parent.parent

def load_docs(folder):
    docs = []
    for path in sorted((ROOT / folder).glob("*.yaml")):
        docs += [d for d in yaml.safe_load_all(path.read_text()) if d]
    return docs

def find(docs, kind, name):
    return next(d for d in docs if d["kind"] == kind and d["metadata"]["name"] == name)

APP = load_docs("k8s")
MON = load_docs("monitoring")

def test_deployment_selector_matches_pod_labels():
    for dep in [d for d in APP + MON if d["kind"] == "Deployment"]:
        selector = dep["spec"]["selector"]["matchLabels"]
        labels = dep["spec"]["template"]["metadata"]["labels"]
        assert selector.items() <= labels.items(), dep["metadata"]["name"]

def test_every_service_selects_a_deployment_and_a_named_port():
    deployments = [d for d in APP + MON if d["kind"] == "Deployment"]
    for svc in [d for d in APP + MON if d["kind"] == "Service"]:
        selector = svc["spec"]["selector"]
        matches = [
            d for d in deployments
            if selector.items() <= d["spec"]["template"]["metadata"]["labels"].items()
        ]
        assert matches, f"Service {svc['metadata']['name']} selects no pods"
        port_names = {p["name"] for c in matches[0]["spec"]["template"]["spec"]["containers"]}
        for p in c["ports"]:
            for port in svc["spec"]["ports"]:
                assert port["targetPort"] in port_names

def test_probes_share_one_health_check():
    container = find(APP, "Deployment", "flask-app")["spec"]["template"]["spec"]["containers"]
    assert container["readinessProbe"]["httpGet"] == container["livenessProbe"]["httpGet"]
    assert container["resources"]["requests"] and container["resources"]["limits"]

def test_prometheus_config_is_valid_yaml_with_app_job():
    config = yaml.safe_load(find(MON, "ConfigMap", "prometheus-config")["data"] ["prometheus."])
```

```

yaml""))
assert "app-pods" in {job["job_name"] for job in config["scrape_configs"]}

def test_dashboard_uses_provisioned_datasource():
    provisioning = find(MON, "ConfigMap", "grafana-provisioning")["data"]
    uid = yaml.safe_load(provisioning["datasources.yaml"])["datasources"][0]["uid"]
    dashboard = json.loads((ROOT / "monitoring/dashboards/flask-app.json").read_text())
    assert {p["datasource"]["uid"] for p in dashboard["panels"]} == {uid}

```